

BCA 5th Semester

PHP

Topic: PHP and MySQL Integration

- By Soudip Sir

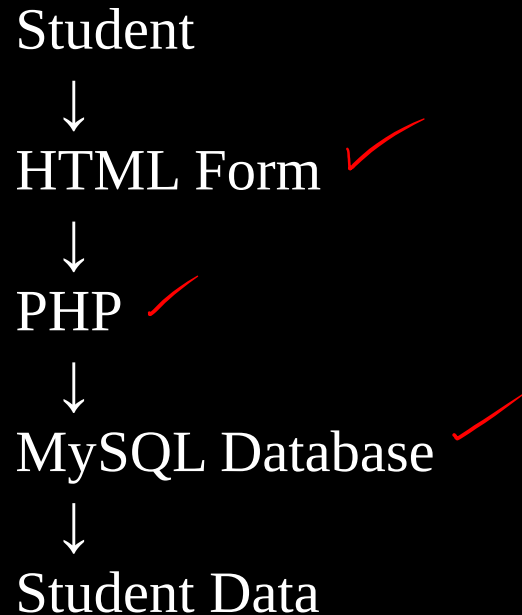
4

PHP and MySQL Integration: Connecting PHP to MySQL, executing queries, retrieving data, and displaying data in web pages Implementing Create, Read, Update, Delete (CRUD) operations using PHP and MySQL

What is PHP and MySQL Integration?

PHP and MySQL integration means connecting a PHP application with a MySQL database so that PHP can communicate with and manage the data stored in MySQL.

For example, consider a student registration website:



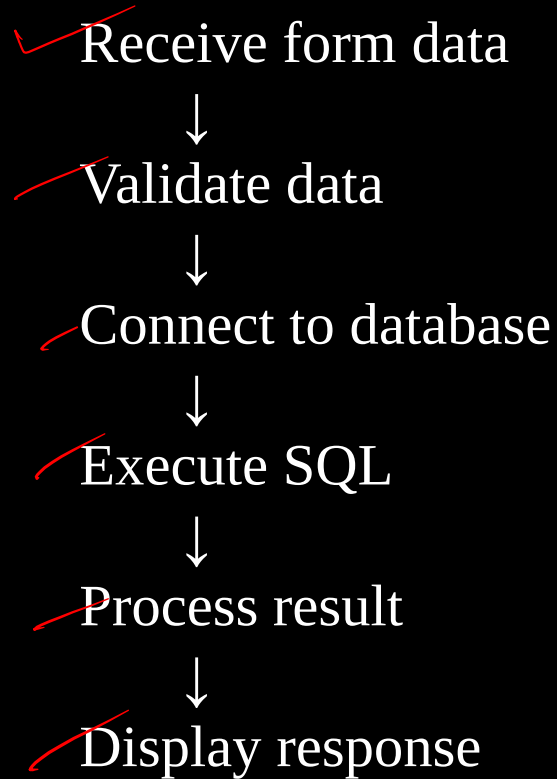
PHP acts as the **middle layer** between the website and the database.

Role of PHP and MySQL

PHP	MySQL
Server-side programming language	Relational database management system
Processes user requests	Stores data
Handles application logic	Manages database records
Connects application to database	Executes SQL queries
Generates dynamic webpages	Provides requested data

What Does PHP Do?

PHP is responsible for the **application logic**.



What Does MySQL Do?

MySQL is responsible for **storing and managing data**.

For example:

Database: college

Tables:

- students

- teachers

- courses

- departments

How PHP connects to MySQL

Connection options

- **PDO (PHP Data Objects)** — recommended: supports many DBs, prepared statements, exceptions.
- **mysqli** — MySQL-only extension, supports prepared statements too (object-oriented or procedural).

Why use PDO?

- Unified API across databases.
- Prepared statements by default (helps prevent SQL injection).
- Easier error handling with exceptions.

Example: PDO connection (with explanation)

// config.php - central DB config (use this file in other scripts)

```
<?php
```

```
$host = 'localhost';
```

```
$db  = 'my_database';
```

```
$user = 'my_user';
```

```
$pass = 'my_password';
```

```
$charset = 'utf8mb4';
```

```
$dsn = "mysql:host=$host;dbname=$db;charset=$charset";
```

```
$options = [
```

```
    PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION, // throw exceptions on errors
```

```
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,      // fetch rows as associative arrays
```

```
    PDO::ATTR_EMULATE_PREPARES => false,                    // use native prepared statements
```

```
];
```

```
try {
```

```
    $pdo = new PDO($dsn, $user, $pass, $options);
```

```
} catch (PDOException $e) {
```

```
    // In production, don't echo credentials or SQL errors — log them instead
```

```
    exit('Database connection failed: ' . $e->getMessage());
```

```
}
```



Explanation

- `$dsn`: connection string (host + db + charset).
- `ATTR_ERRMODE` set to `EXCEPTION` so you can catch DB errors.
- `ATTR_DEFAULT_FETCH_MODE` → `FETCH_ASSOC` returns associative arrays.
- `EMULATE_PREPARES` = `false` instructs PDO to use DB-native prepared statements (more secure).

Connecting PHP to MySQL (Create: db.php)

```
<?php
```

```
$conn = new mysqli(  
    "localhost", ✓  
    "root", ✓  
    "", ✓  
    "college" ✓  
);
```

```
if ($conn->✓connect_error) {  
    die("Connection failed: " . $conn->connect_error);  
}
```

```
echo "Database connected successfully!";  
?>
```



Don't Repeat the Connection Code

Instead of writing this in every PHP file:

```
</> PHP  
  
$conn = new mysqli(...);
```

we can create one file:

```
db.php
```

and include it wherever required.

For example:

```
</> PHP  
  
require_once "db.php";
```

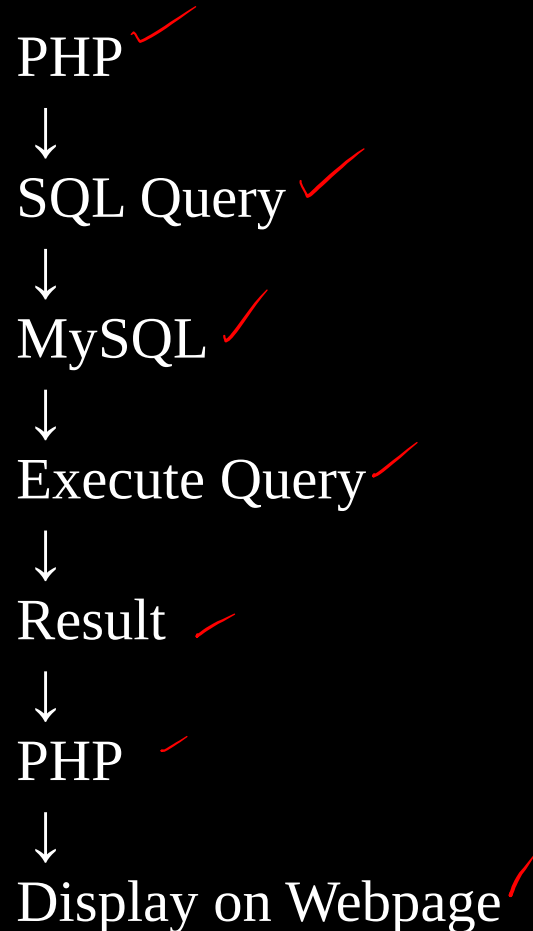


This is one of the practical uses of the `require_once` concept from your previous module.

Executing Queries and Retrieving Data in PHP + MySQL

After connecting PHP to MySQL, the next important step is learning how to **send SQL queries to MySQL** and **retrieve the results returned by MySQL**.

The basic process is:



What is a Query?

A **query** is an SQL command sent to the database to perform an operation.

Executing a Query Using query() ✓

```
$conn->query($sql);
```

SELECT Query

```
✓ $sql = "SELECT * FROM students"; ✓
```

```
✓ $result = $conn->query($sql);
```

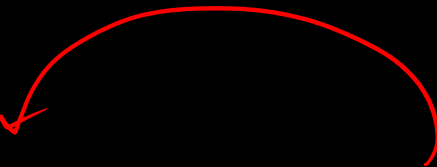
What is \$result?

- \$result contains the result returned by MySQL for a successful SELECT.
- It is not the actual array of students yet.
- We need to fetch the rows from the result.

Retrieving Data Using fetch_assoc()

We use:

```
$result->fetch_assoc();
```



```
while ($row = $result->fetch_assoc()) {  
    echo $row["name"] . "<br>";  
}
```

Complete Example — Retrieve Students

```
<?php
```

```
$conn = new mysqli(  
    "localhost",  
    "root",  
    "",  
    "college"  
);  
  
if ($conn->connect_error) {  
    die("Connection failed: " . $conn->connect_error);  
}  
  
$sql = "SELECT * FROM students";  
  
$result = $conn->query($sql);
```

```
while ($row = $result->fetch_assoc()) {  
  
    echo "ID: " . $row["id"] . "<br>";  
    echo "Name: " . $row["name"] . "<br>";  
    echo "Email: " . $row["email"] . "<br>";  
    echo "Age: " . $row["age"] . "<br>";  
  
    echo "<hr>";  
}  
  
?>
```

Displaying Retrieved Data in an HTML Table

```
<?php  
  
require_once "db.php";  
  
$sql = "SELECT * FROM students";  
  
$result = $conn->query($sql);  
  
?>
```

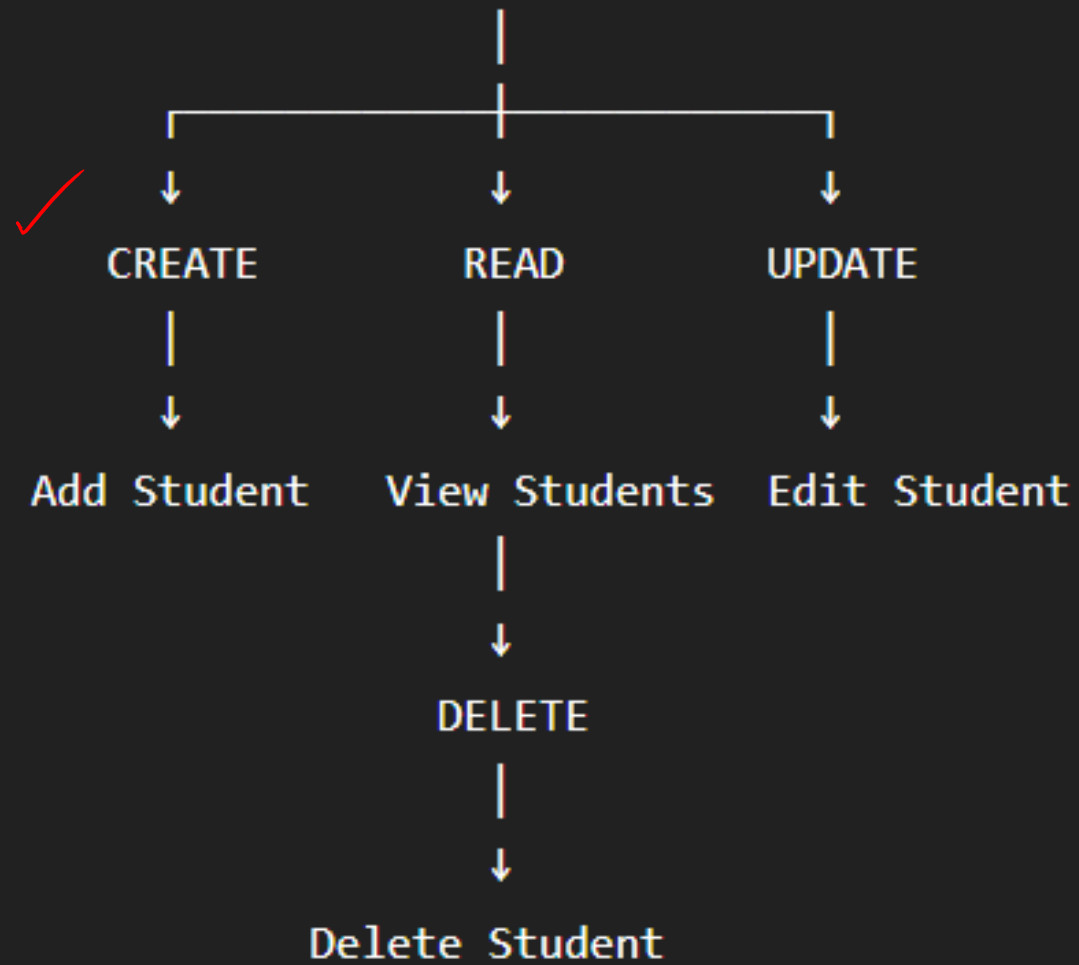
```
<table border="1">
```

```
    <tr>  
        <th>ID</th>  
        <th>Name</th>  
        <th>Email</th>  
        <th>Age</th>  
    </tr>
```

```
<?php  
while ($row = $result->fetch_assoc()) {  
    ?>  
    <tr>  
        <td>  
            <?= htmlspecialchars($row["id"]) ?>  
        </td>  
  
        <td>  
            <?= htmlspecialchars($row["name"]) ?>  
        </td>  
  
        <td>  
            <?= htmlspecialchars($row["email"]) ?>  
        </td>  
  
        <td>  
            <?= htmlspecialchars($row["age"]) ?>  
        </td>  
  
    </tr>  
  
    <?php  
    }  
    ?>  
}</table>
```

Displaying Data in Web Pages & CRUD Operations Using PHP and MySQL

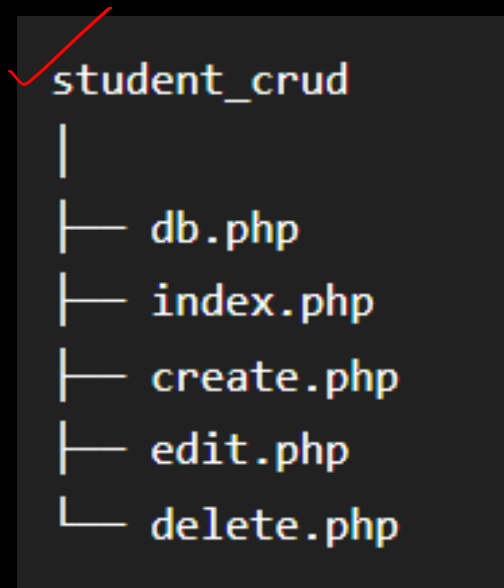
STUDENT MANAGEMENT SYSTEM



What is CRUD?

CRUD represents the four basic operations performed on database records.

Letter	Operation	SQL Command	Example
C	Create ✓	INSERT	Add a student
R	Read ✓	SELECT	Display students
U	Update ✓	UPDATE	Edit student
D	Delete ✓	DELETE	Remove student



File	Purpose
db.php	Database connection
index.php	Display all students
create.php	Add a new student
edit.php	Update student
delete.php	Delete student

Why header("Location: index.php")?

After successfully adding the student:header("Location: index.php");
exit;

sends the user back to the student list.

✓ How fetch_assoc() Works

\$row = \$result->fetch_assoc();
we can access:

\$row["id"]
\$row["name"]
\$row["email"]
\$row["age"]

What is <?= ?> ?

```
<?php echo $row["name"]; ?>
```

means:

Print the value of \$row["name"].

Why htmlspecialchars() ?

We use:

```
htmlspecialchars($row["name"])
```

when putting database/user-controlled text into HTML.

For example:

```
<?= htmlspecialchars($row["name"]) ?>
```

This ensures special HTML characters are encoded rather than interpreted as HTML.

Understanding `prepare()`

We use:

`</>` **PHP**

```
$stmt = $conn->prepare(  
    "INSERT INTO students (name, email, age)  
    VALUES (?, ?, ?)"  
);
```

Notice:

```
?  
?  
?
```

These are **placeholders**.

We don't directly insert user input into the SQL statement.

Understanding `bind_param()`

`</>` PHP

```
$stmt->bind_param(  
    "ssi",  
    $name,  
    $email,  
    $age  
);
```

The first argument:

`"ssi"`

represents the data types:

```
s → string  
s → string  
i → integer
```

Why Prepared Statements?

Never do this with untrusted input:

`</>` **PHP**

```
$sql = "INSERT INTO students  
VALUES ('$name', '$email', '$age')";
```

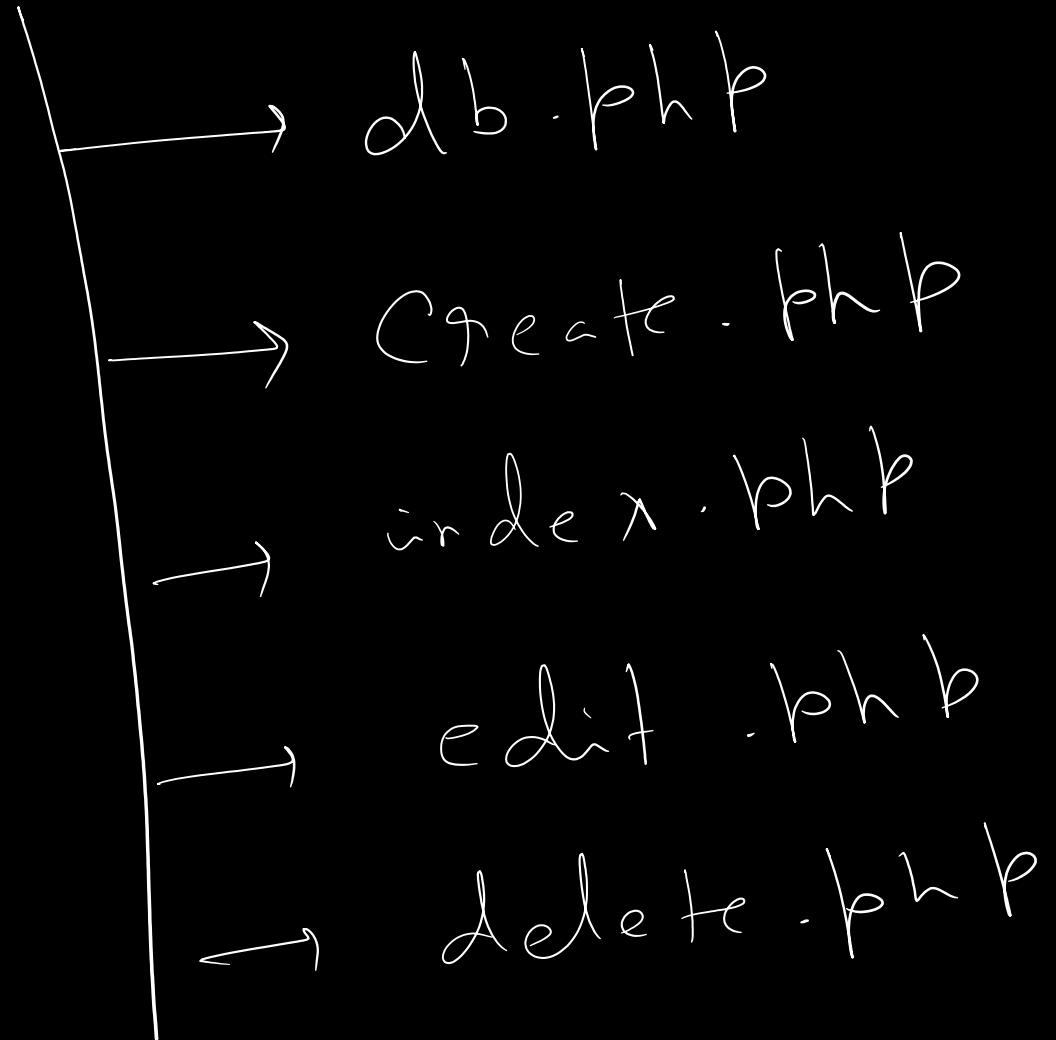
Instead, use:

`</>` **PHP**

```
prepare()  
bind_param()  
execute()
```

Prepared statements help protect SQL queries from **SQL injection**.

GH - Student6



1> Well designed Table ✓ (7-8)

2> CREATE

3> READ

4> UPDATE

(2) video
✓